



Laboratorio No.2 Semana #13 – API Secrets y autenticación

Nombre: _Kenneth Isafás Yahid López Castillo_____ **Carné:** _1138025_____

Objetivo: Practicar el consumo de una API REST autenticada y aplicar una buena práctica básica de seguridad: evitar colocar credenciales directamente en el código fuente. El estudiante utilizará Postman, una aplicación C# y GitHub.

Parte 1 – Probar autenticación con Postman

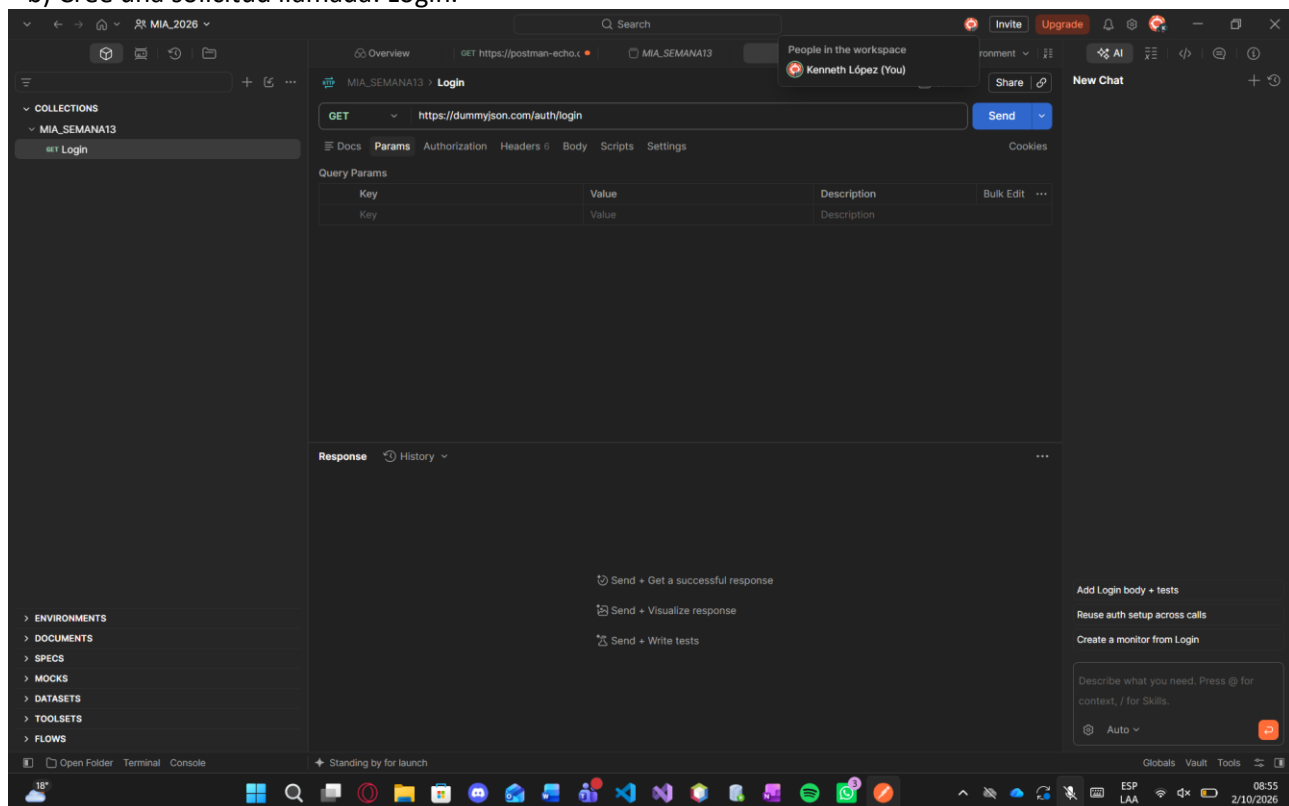
Para esta práctica se utilizará DummyJSON, una API pública de pruebas que permite realizar un inicio de sesión y obtener un token de acceso.

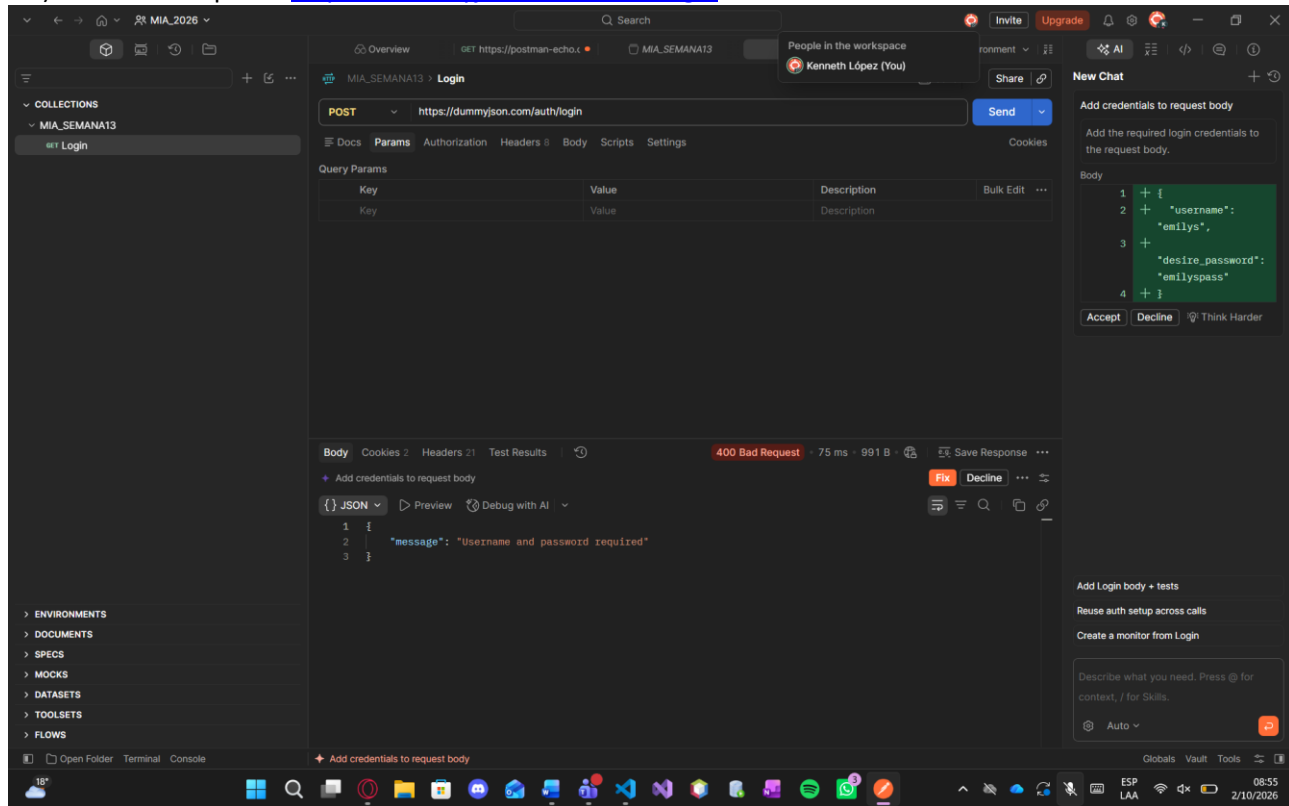
Documentación: <https://dummyjson.com/docs/auth> <https://dummyjson.com/docs/users>

1. Crear la solicitud de inicio de sesión

a) Abra Postman y cree una nueva colección llamada: MIA – Semana 13.

b) Cree una solicitud llamada: Login.



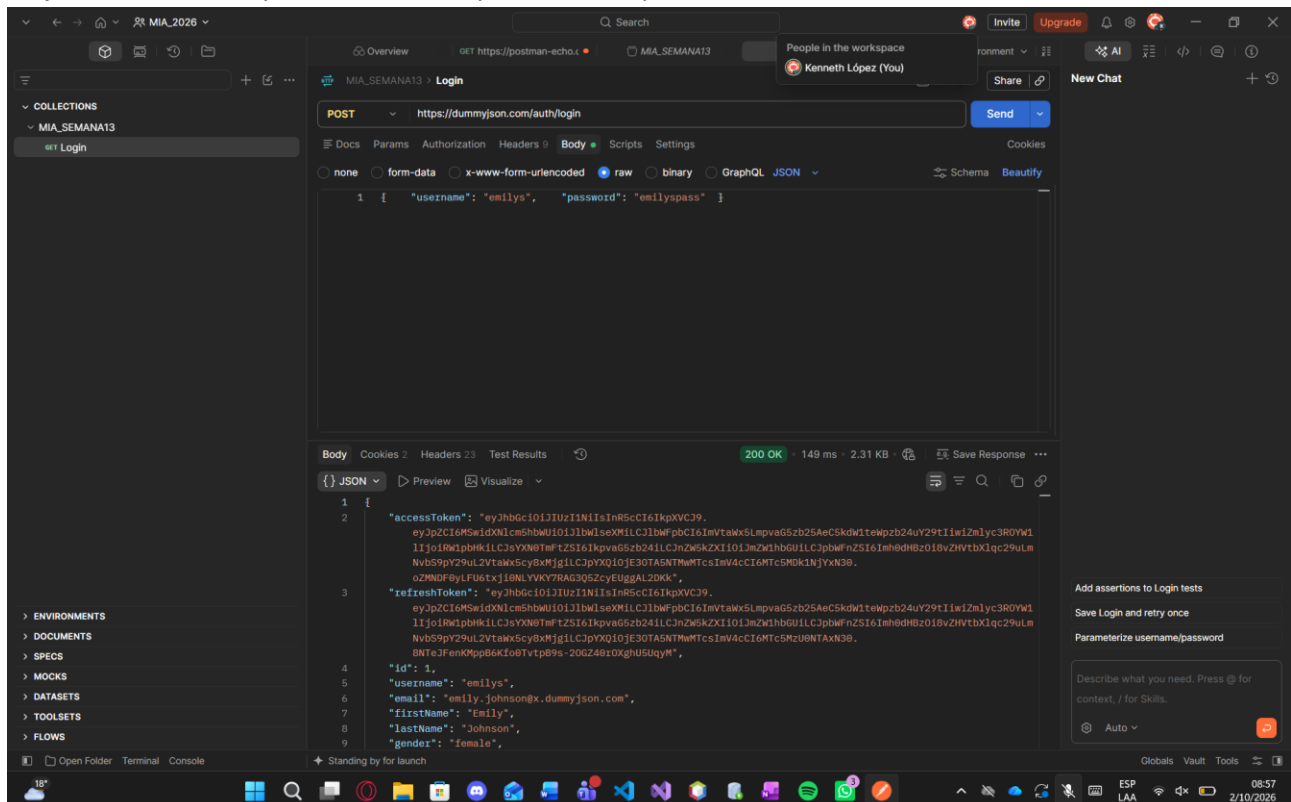


e) En Body seleccione raw y JSON.

Ingrese las credenciales de prueba: Username: emilys | Password: emilyspass.

```
{ "username": "emilys", "password": "emilyspass" }
```

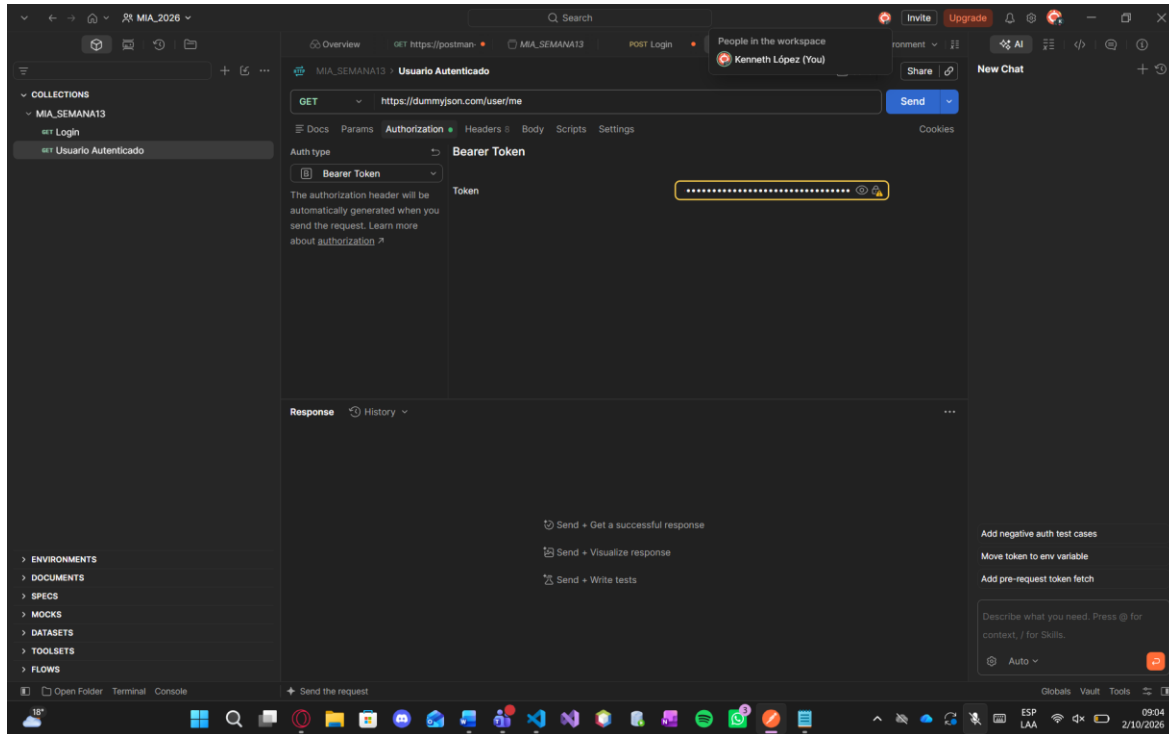
f) Ejecute la solicitud y localice en la respuesta el campo accessToken.



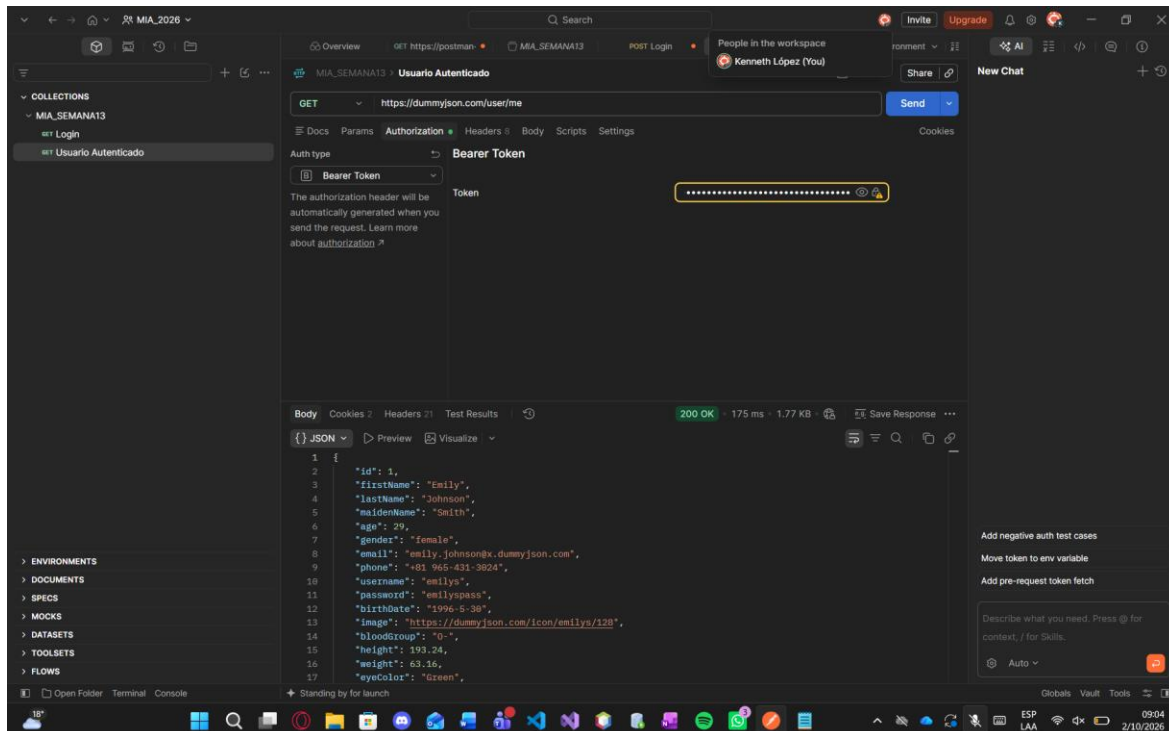


2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: <https://dummyjson.com/user/me>
- En Authorization seleccione Bearer Token.
- Copie el accessToken obtenido en la solicitud Login.



- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.





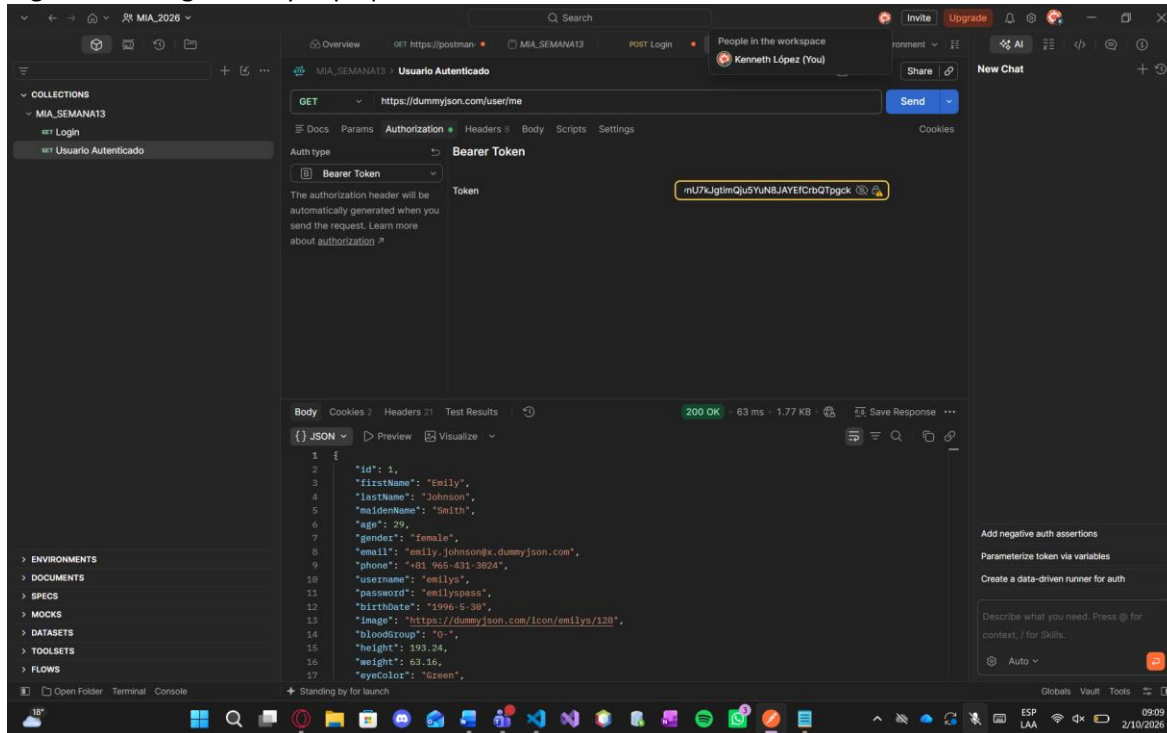
3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.

The screenshot shows the Postman interface with a collection named 'MIA_SEMANA13'. The selected request is 'Usuario Autenticado' (GET https://dummyjson.com/user/me). The 'Auth type' is set to 'Bearer Token'. The response is a 401 Unauthorized status with a message: "Invalid/Expired Token!". The 'Fix authentication' button is visible in the top right corner.

d) Compare la respuesta con la solicitud autenticada.

The screenshot shows the Postman interface with the same collection 'MIA_SEMANA13'. The selected request is 'Login' (POST https://dummyjson.com/auth/login). The 'Body' tab is selected, showing a JSON payload: {"username": "emilys", "password": "emilyspass"}. The response is a 200 OK status with a JSON body containing user details and tokens, including 'accessToken', 'refreshToken', 'id', 'username', 'email', 'firstName', 'lastName', and 'gender'.



El token es temporal por seguridad, así que se el token generado primero se cancela y ya no deja por autorización logearse. Se debe generar otra solicitud para crear un nuevo token y volver a intentar.

4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.



Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

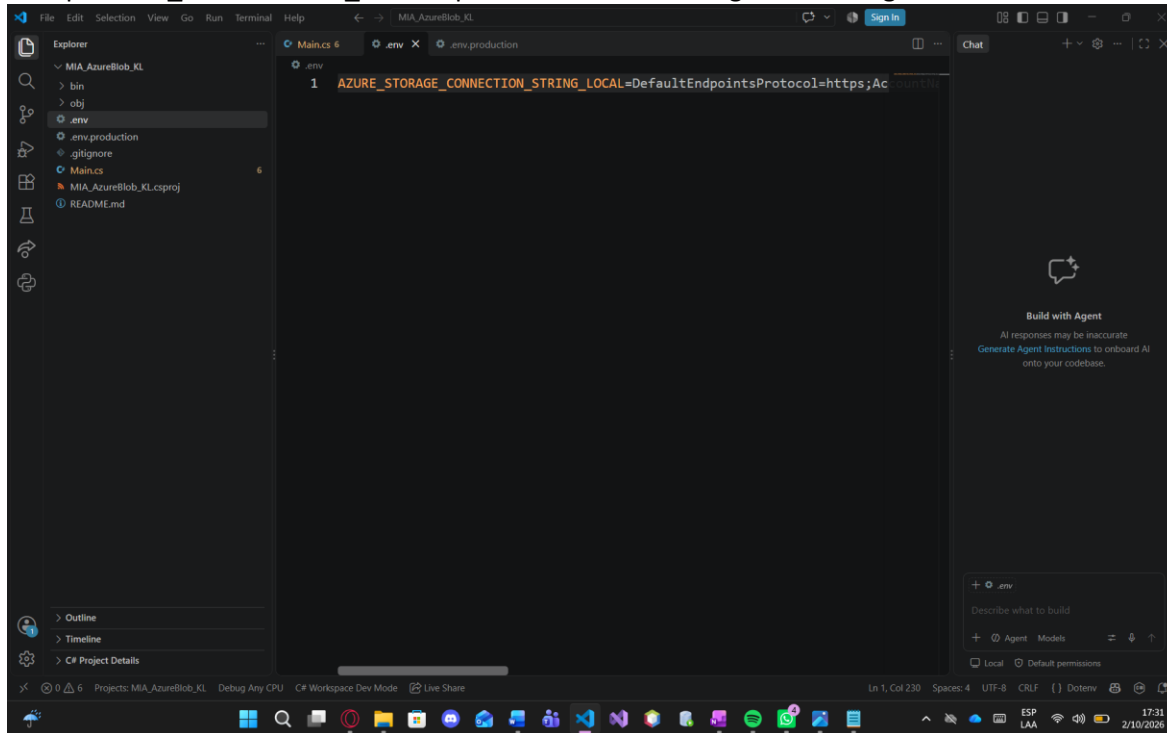
6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada: **AZURE_STORAGE_CONNECTION_STRING**
La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

```
8 class Program
9 {
10     1 reference
11     private static string containerName = "miaarchivos";
12
13     0 references
14     static async Task Main(string[] args)
15     {
16         Env.Load();
17
18         string? connectionString =
19             Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
20
21         if (string.IsNullOrEmpty(connectionString))
22         {
23             Console.WriteLine("No se encontro la Connection String.");
24             Console.WriteLine("Configure la variable AZURE_STORAGE_CONNECTION_STRING");
25             return;
26         }
27
28         BlobServiceClient blobServiceClient = new BlobServiceClient(connectionString);
29         BlobContainerClient containerClient = blobServiceClient.GetBlobContainerClient(containerName);
30
31         // Crear el contenedor si aún no existe en Azure
32         await containerClient.CreateIfNotExistsAsync();
33
34         bool salir = false;
35
36         while (!salir)
37         {
38             Console.Clear();
39             Console.WriteLine("=====");
40         }
41     }
42 }
```

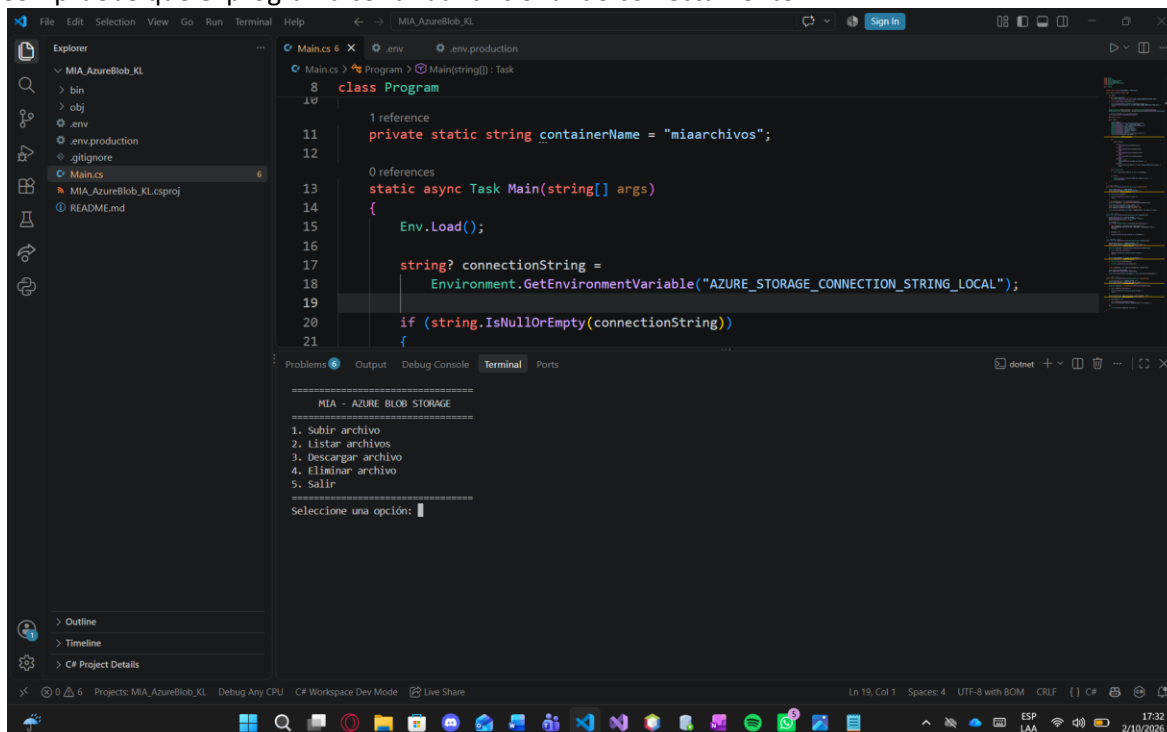
7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.



```
1 AZURE_STORAGE_CONNECTION_STRING_LOCAL=DefaultEndpointsProtocol=https;AccountName=;AccountKey=;EndpointSuffix=;
```

- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.



```
8 class Program
9 {
10     1 reference
11     private static string containerName = "misarchivos";
12
13     0 references
14     static async Task Main(string[] args)
15     {
16         Env.Load();
17
18         string? connectionString =
19             Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING_LOCAL");
20         if (string.IsNullOrEmpty(connectionString))
21         {
22
23         }
24     }
25 }
```

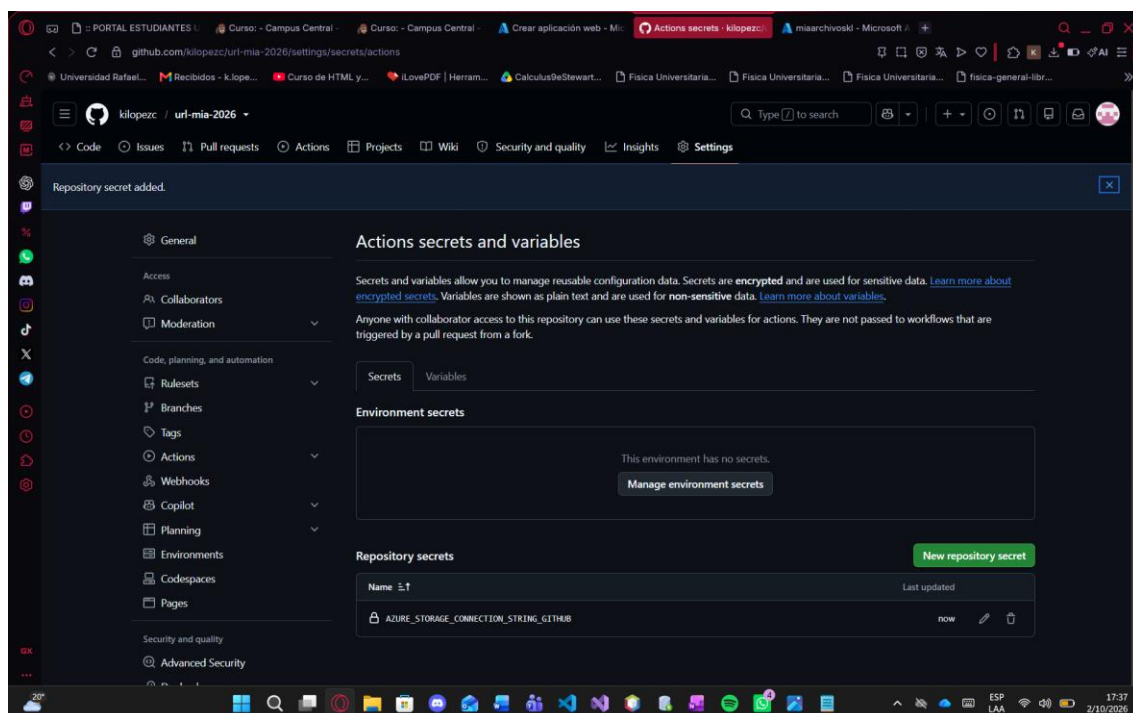
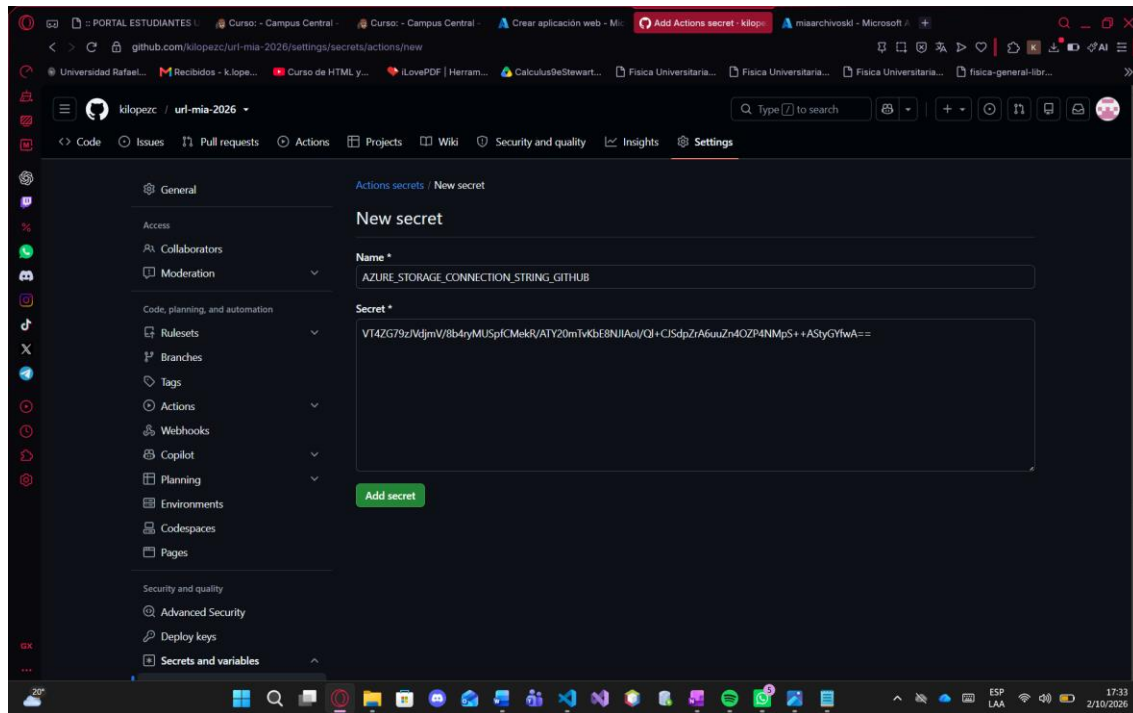
```
MIA - AZURE BLOB STORAGE
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
Seleccione una opción: 1
```

- No publique el archivo .env en GitHub, ya que contiene la credencial real.



8. Crear el Secret en Github

- Ingrese al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: AZURE_STORAGE_CONNECTION_STRING
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.





Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.